自进化智能体

核心架构与算法解析

作者：Yang kunpeng

Program Evolution、Agent Evolution 与 Recursive Self-Improvement

基于 10 篇论文源码的图解与算法导读

目录

第一章 Program Evolution：程序进化 **1**

1.1 AlphaEvolve：面向科学与算法发现的编码智能体 1

1.2 ShinkaEvolve：开放式且具样本效率的程序进化 3

第二章 Agent Evolution：智能体进化 **6**

2.1 EvoAgentX：自动进化智能体工作流 6

2.2 Agent0：从零外部数据出发的双智能体协同进化 7

2.3 AgentEvolver（2025）：任务生成、经验引导与步骤归因 10

第三章 Recursive Self-Improvement：递归自我改进 **12**

3.1 Gödel Agent：自指式智能体与运行时自修改 12

3.2 SICA：自我改进的 Coding Agent 15

3.3 Darwin Gödel Machine：开放式自我进化 17

3.4 Huxley-Gödel Machine：以进化潜力指导搜索 19

3.5 HyperAgents：让改进机制本身成为可修改对象 21

第 1 章

Program Evolution：程序进化

1.1 AlphaEvolve：面向科学与算法发现的编码智能体

论文： AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery（arXiv:2506.13131）

开源参考： OpenEvolve，约 7.5k Stars（2026-10-17）；社区非官方实现。

核心架构图与解释

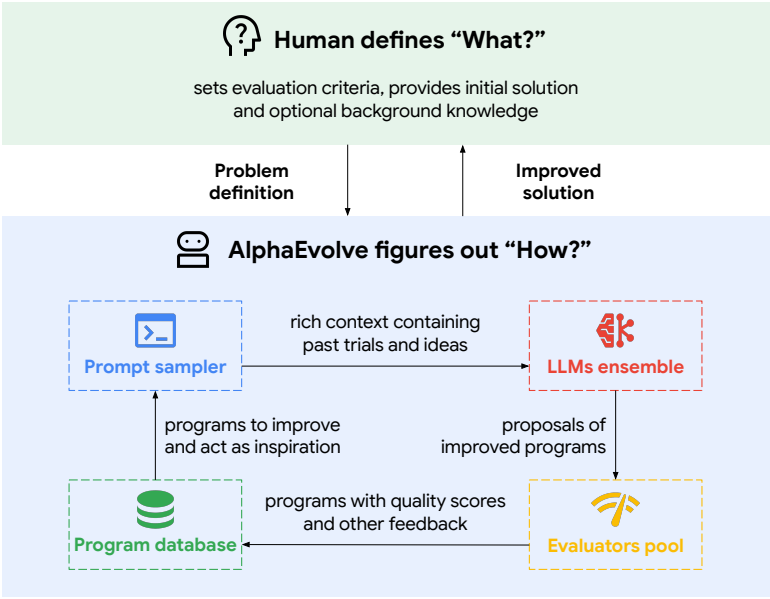


图 1.1: AlphaEvolve 的高层流程。

原图来源：论文 arXiv:2506.13131。

AlphaEvolve 将程序发现组织成一个由大语言模型驱动的进化循环。用户提供初始程序和自动评价函数，并可标记程序中允许修改的代码块；其余代码构成固定骨架，负责连接这些代码块并调用评价流程。系统反复提出候选修改、运行评价，再利用结果指导后续搜索。

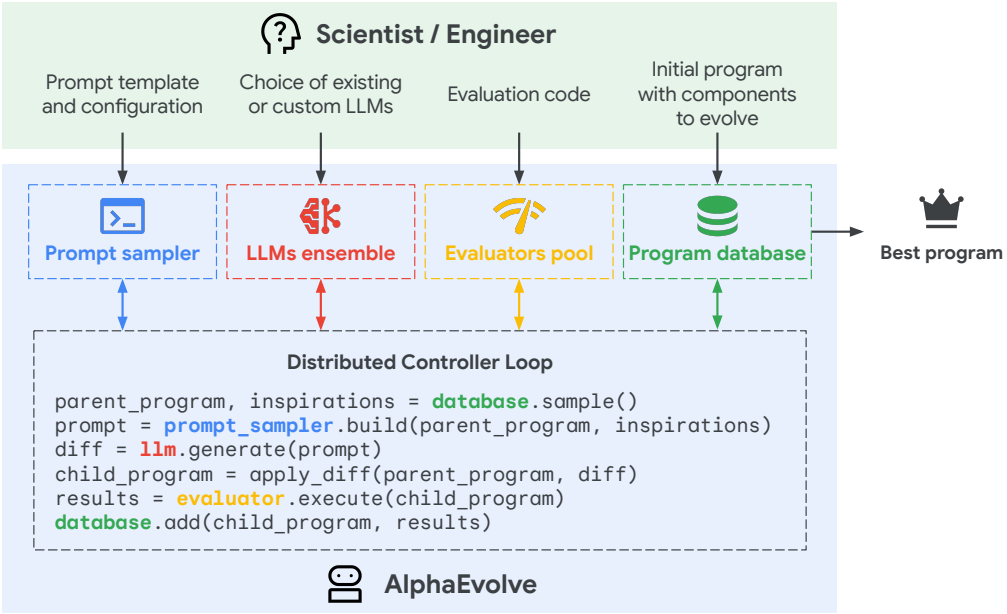


图 1.2: AlphaEvolve 的程序发现流程。

原图来源：论文 arXiv:2506.13131。

详细架构将循环拆成几个相互衔接的环节：

1. **Prompt sampler**。从 Program database 选取已有程序及其评价结果，构造候选生成提示。提示可结合任务约束、历史方案和其他相关上下文。
2. **LLM ensemble**。一个或多个语言模型提出代码修改。论文使用 SEARCH/REPLACE 形式的 diff，让修改定位到程序中的具体片段。
3. **Patch application**。将修改应用到当前程序；无法匹配或不符合修改约束的补丁不能直接进入评价阶段。
4. **Evaluators**。运行用户提供的评价函数，返回一个或多个指标。评价函数定义任务目标，可以检查解是否满足约束，也可以运行搜索程序或训练模型后给出分数。

5. **Program database**。保存候选程序及其评价结果，供后续采样和提示构造使用。数据库保留的不只是当前最佳方案，因此搜索可以从不同的历史方案继续展开。

这套架构将候选生成、程序执行与评价、历史方案复用分开处理。语言模型负责提出修改，评价函数提供任务反馈，程序数据库则把反馈与候选关联起来，形成持续迭代的搜索过程。

1.2 ShinkaEvolve：开放式且具样本效率的程序进化

论文：ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution（arXiv:2509.19349）

开源参考：SakanaAI/ShinkaEvolve，约 1.5k Stars（2026-10-17）。

核心架构图与解释

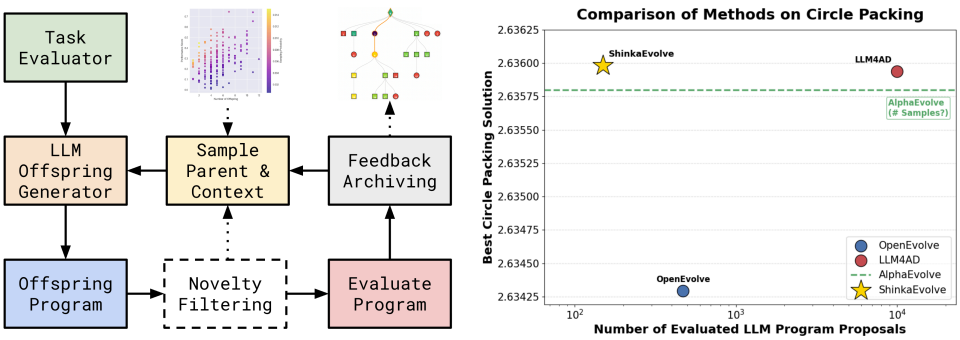


图 1.3: ShinkaEvolve 的整体进化循环。

原图来源：论文 arXiv:2509.19349。

ShinkaEvolve 使用多岛进化（island model）组织程序档案。每个岛维护独立的程序群体，并行探索不同的修改方向；父代选择会同时考虑程序表现和探索需求，部分历史程序还会作为 inspiration programs 提供给模型参考。岛之间可以迁移候选，但保留各岛的优良方案，以减少搜索过早集中到同一分支的情况。

候选可通过 diff 局部编辑、受修改区域约束的完整重写，或融合不同档案程序的 crossover 产生。生成后先检查补丁和不可修改区域，再进行新颖性筛选；通过筛选的候选才会执行并评价。

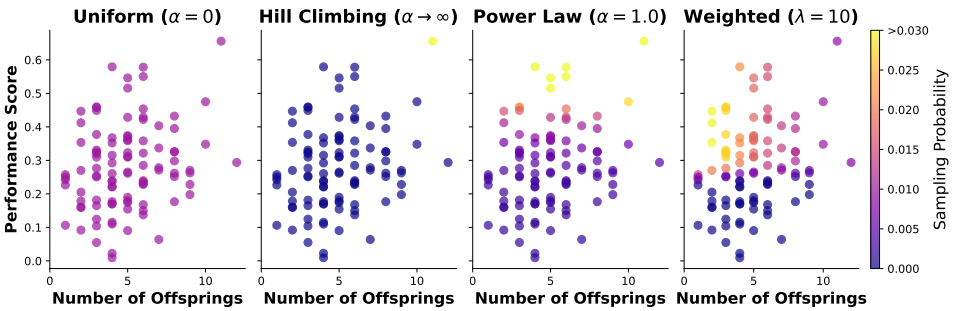


图 1.4: ShinkaEvolve 的父代采样策略。

原图来源：论文 arXiv:2509.19349。

数学表示

论文提供了两类父代选择方式。幂律排名采样以程序排名 r_i 决定被选中的概率：

$$p_i = \frac{r_i^{-\alpha}}{\sum_{j=1}^{n} r_j^{-\alpha}}. \tag{1.1}$$

其中最优程序排名为 1。当 $\alpha = 0$ 时，各程序被均匀采样；随着 α 增大，选择会更集中于排名靠前的程序。

另一种加权采样同时考虑程序表现和其已有后代数量。令 $F(P_i)$ 为程序适应度，$N(P_i)$ 为它已有的后代数，α_0 为群体适应度中位数，则

$$s_i = \sigma(\lambda(F(P_i) - \alpha_0)), \tag{1.2}$$

$$h_i = \frac{1}{1 + N(P_i)}, \qquad w_i = s_i h_i, \tag{1.3}$$

$$p_i = \frac{w_i}{\sum_{j=1}^{n} w_j}. \tag{1.4}$$

s_i 提高表现优于群体中位数的程序被选中的机会；h_i 则降低已产生大量后代的程序的权重。二者相乘，使父代选择同时利用已知表现和探索新分支的需要。

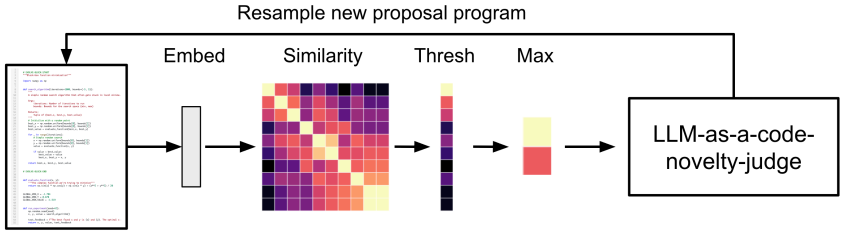


图 1.5: 基于代码嵌入与 LLM 判断的新颖性拒绝采样。

原图来源：论文 arXiv:2509.19349。

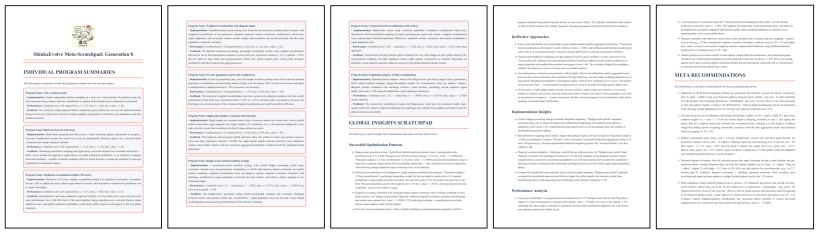


图 1.6: Meta-scratchpad 的信息组织。

原图来源：论文 arXiv:2509.19349。

新颖性拒绝采样用于减少重复探索：代码嵌入先筛查与现有程序过于相似的候选，再由语言模型判断其是否包含实质性的新想法。Meta-scratchpad 则把单个程序的摘要、跨程序的共性观察和实现建议整理成提示上下文，使后续修改能够利用先前搜索积累的经验。

系统还会记录候选的评价指标与文本反馈，并根据候选相对于父代和初始程序的改进情况调整不同 LLM 的采样概率。由此，候选生成、程序评价、新颖性筛选和经验积累共同影响后续搜索。

第 2 章

Agent Evolution：智能体进化

2.1 EvoAgentX：自动进化智能体工作流

论文：EvoAgentX: An Automated Framework for Evolving Agentic Workflows（arXiv:2507.03616）

开源参考：ANative-Lab/EvoAgentX，约 3.4k Stars（2026-10-17）。

核心架构图与解释

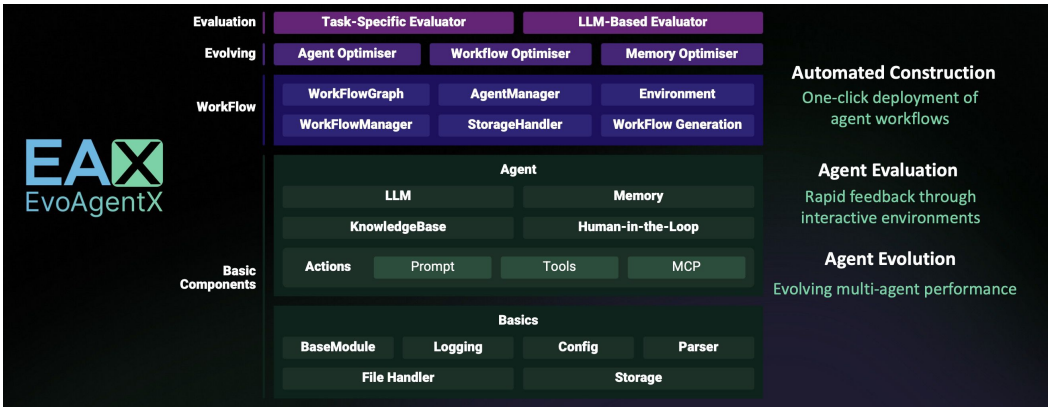


图 2.1: EvoAgentX 的分层系统架构。

原图来源：论文 arXiv:2507.03616。

图中的五层从基础服务延伸到任务执行，再连接评价与进化。Basic Component Layer 提供配置、日志、文件处理和存储；Agent Layer 将 LLM、Memory 与 Actions 组合成可执行的智能体；Workflow Layer 用图结构组织任务节点及其依赖；Evaluation Layer 根据任务指标或 LLM 评价器评估执行结果；Evolving Layer 则利用反馈更新 Agent 配置、工作流结构或记忆策略。

因此，EvoAgentX 不只运行工作流，也提供修改工作流的外层机制：工作流执行产生结果，评价层将结果转成反馈，优化器据此生成下一版 Agent 配置或工作流。工作流可以表示为一般有向图，也可以根据输入输出依赖构造成线性流程。

数学表示

论文将单个 Agent 表示为 LLM、记忆模块和动作集合的组合：

$$a_i = \left\langle \mathrm{LLM}_i, \mathrm{Mem}_i, \{\mathrm{Act}_i^{(j)}\}_{j=1}^{M} \right\rangle. \tag{2.1}$$

工作流表示为有向图：

$$\mathcal{W} = (\mathcal{V}, \mathcal{E}), \tag{2.2}$$

其中节点 $\mathcal{V}$ 表示任务，边 $\mathcal{E}$ 表示任务依赖和数据流。节点可包含一个或多个 Agent，也可以封装显式的动作序列。

Evolving Layer 中有三个优化方向。Agent optimizer 更新提示词、工具配置和动作策略；Workflow optimizer 修改图中的节点、边、顺序与依赖；Memory optimizer 的目标是调整记忆的保留、剪枝和检索策略。论文将记忆优化描述为仍在开发中的组件，不能把它视为已完全成熟的固定算法。

相应的更新过程在论文中表示为：

$$(\mathrm{Prompt}_i^{(t+1)}, \theta_i^{(t+1)}) = \mathcal{O}_{\mathrm{agent}}(\mathrm{Prompt}_i^{(t)}, \theta_i^{(t)}, \mathcal{E}), \tag{2.3}$$

$$\mathcal{W}^{(t+1)} = \mathcal{O}_{\mathrm{workflow}}(\mathcal{W}^{(t)}, \mathcal{E}), \tag{2.4}$$

$$\mathcal{M}_i^{(t+1)} = \mathcal{O}_{\mathrm{memory}}(\mathcal{M}_i^{(t)}, \mathcal{E}). \tag{2.5}$$

这里的 $\mathcal{E}$ 表示评价反馈，不是环境状态。三个算子分别作用于 Agent 配置、工作流拓扑和记忆结构，体现了不同层级的优化边界。

2.2 Agent0：从零外部数据出发的双智能体协同进化

论文： Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning（arXiv:2511.16043）

开源参考： aiming-lab/Agent0，约 1.3k Stars（2026-10-17）。

核心架构图与解释

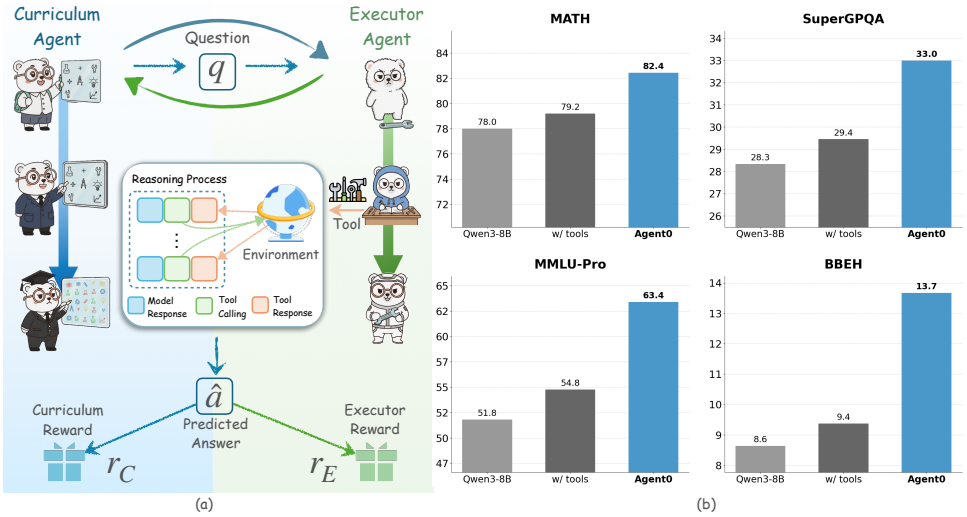


图 2.2: Agent0 系列的迭代进化视图。

原图来源：论文 arXiv:2511.16043。

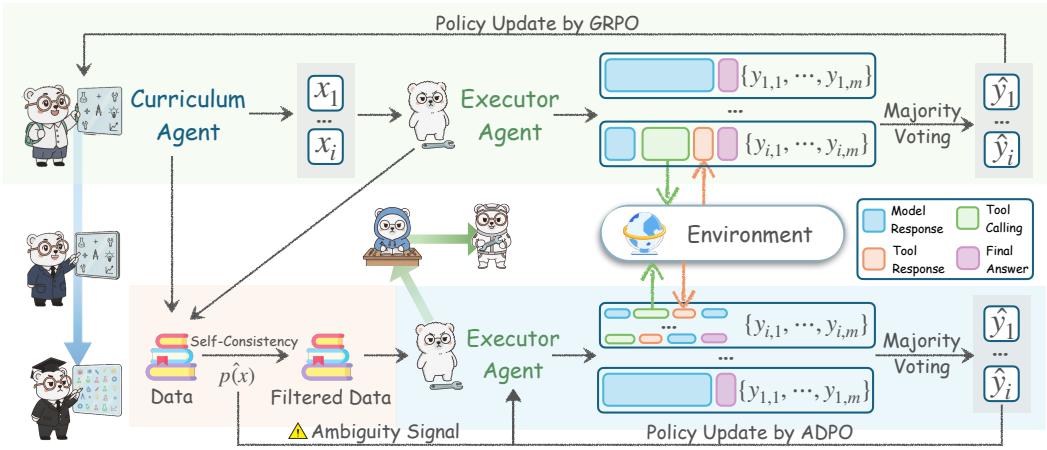


图 2.3: Curriculum Agent 与 Executor Agent 的协同训练机制。

原图来源：论文 arXiv:2511.16043。

Agent0 由两个职责不同的智能体组成。Curriculum Agent 生成训练任务，目标是找到当前 Executor Agent 尚不能稳定解决、但又具有学习价值的任务；Executor Agent 则通过多步推理和工具调用完成任务。两者交替更新：先训练 Curriculum

Agent，再固定它并生成候选任务池，根据 Executor 的回答一致性筛选出难度适中的任务，最后用筛选后的任务训练 Executor。Executor 的能力变化又会影响下一轮任务生成，形成双智能体的协同进化。

数学表示

论文用多次回答的一致性估计任务对 Executor 的难度。若 k 个采样回答中，最多人支持的答案占比为 $\hat{p}(x)$，则不确定性奖励为

$$R_{\text{unc}}(x;\pi_\phi) = 1 - 2\left|\hat{p}(x;\pi_\phi) - 0.5\right|. \tag{2.6}$$

当答案一致性接近 0.5 时，奖励较高；当多数回答几乎完全一致，或答案极度分散时，奖励较低。该奖励让 Curriculum Agent 倾向于提出具有挑战性、但并非毫无结构的任务。

工具使用奖励与重复惩罚共同约束任务生成。论文将工具调用次数记为 $N_{\text{tool}}(y)$，并使用带上限的奖励：

$$R_{\text{tool}}(x;\pi_\phi) = \gamma \min(N_{\text{tool}}(y), C). \tag{2.7}$$

其中 C 限制过度调用工具带来的奖励。重复惩罚依据批次中相似任务的聚类规模计算；最终课程奖励还受到输出格式检查的门控：

$$R_C(x_i) = R_{\text{format}}(x_i)\max\left(0, \lambda_{\text{unc}}R_{\text{unc}} + \lambda_{\text{tool}}R_{\text{tool}} - R_{\text{rep}}(x_i)\right). \tag{2.8}$$

Executor 的任务数据不是直接从生成池全部取用，而是保留一致性落在指定区间内的任务：

$$\mathcal{D}^{(t)} = \left\{x \in X_{\text{pool}} : \left|\hat{p}(x;\pi_\phi^{(t-1)}) - 0.5\right| \le \delta\right\}. \tag{2.9}$$

图中下半部分显示了工具反馈进入多轮轨迹的方式。Executor 生成一段推理后，可以输出 Python 代码；代码在沙箱中运行，结果或错误再返回模型上下文。模型据此继续推理，直到给出最终答案。训练时，Agent0 还根据答案一致性构造伪标签，并使用 ADPO 调整优势权重和策略更新约束，减轻高歧义任务中伪标签噪声带来的影响。

2.3 AgentEvolver（2025）：任务生成、经验引导与步骤归因

论文：AgentEvolver: Towards Efficient Self-Evolving Agent System（arXiv:2511.10395）

开源参考：modelscope/AgentEvolver，约 1.6k Stars（2026-10-17）。

核心架构图与解释

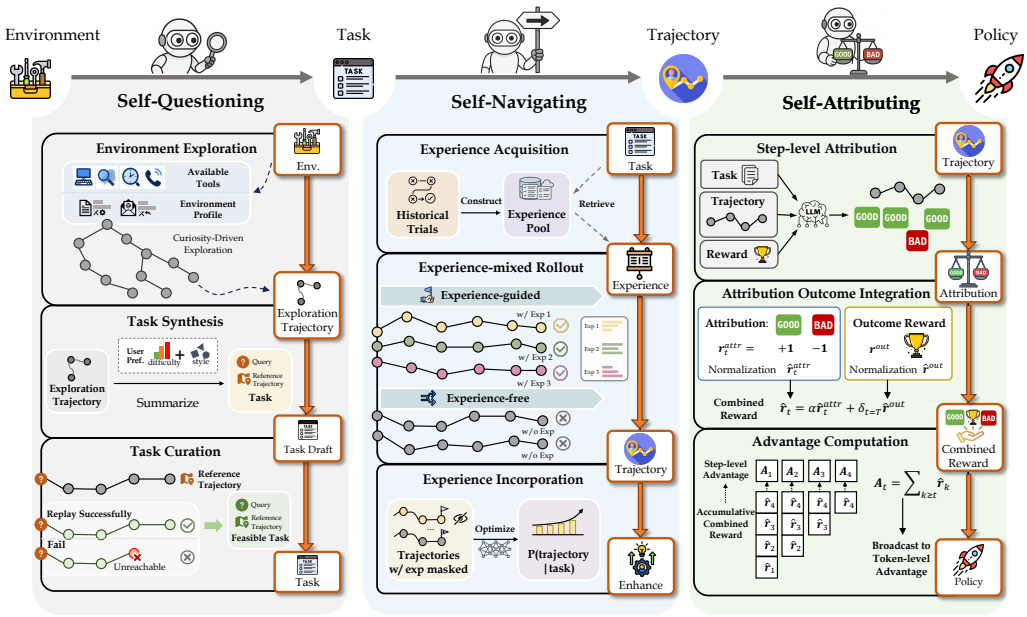


图 2.4: AgentEvolver 的三种协同机制：任务生成、经验引导探索和步骤归因。

原图来源：论文 arXiv:2511.10395。

AgentEvolver 将智能体的自进化组织为从环境到任务、从任务到轨迹、再从轨迹到策略的过程。三个机制分别处理不同的瓶颈。

- **Self-questioning**。 Agent 主动探索环境的状态—动作空间，生成任务并检查任务是否有效，降低对人工编写训练任务的依赖。
- **Self-navigating**。系统从历史执行中提取经验，并把经验用于后续探索。经验可用于构造更有针对性的轨迹引导，而不是每次都从零开始采样。

- **Self-attributing**。对长轨迹中的中间步骤进行质量归因，将过程质量信号与最终结果奖励结合，使策略更新不必只依赖稀疏的终局奖励。

数学表示

Self-attributing 先让评价模型为每个步骤给出 GOOD 或 BAD 标签，并分别映射为 $+1$ 和 -1。经标准化后得到步骤级归因奖励 $\hat{r}_t^{\text{attr}}$。最终奖励将过程归因与终局结果分开标准化，再按权重融合：

$$\hat{r}_t = \alpha \hat{r}_t^{\text{attr}} + \mathbf{1}_{t=T} \hat{r}^{\text{out}}. \tag{2.10}$$

$\mathbf{1}_{t=T}$ 只在轨迹最后一步取值为 1，因此终局结果奖励只加到最后一步；过程归因则为每一步提供较密集的反馈。论文随后使用未折扣的未来奖励和计算步骤优势：

$$A_t = \sum_{k=t}^{T} \hat{r}_k. \tag{2.11}$$

步骤级优势会映射到该步骤生成的 token，用于策略梯度更新。α 控制过程质量与最终任务结果的相对权重。

图中的三种机制首尾相接：Self-questioning 产生任务，任务执行形成轨迹；Self-navigating 从轨迹中积累可复用经验，Self-attributing 则为中间步骤提供质量信号。过程奖励与终局奖励共同用于策略更新。因此，该方法不仅调整任务或提示，还通过强化学习更新 Agent 的策略参数。

第 3 章

Recursive Self-Improvement：递归自我改进

3.1 Gödel Agent：自指式智能体与运行时自修改

论文：Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement（arXiv:2410.04444）
开源参考：Arvid-pku/Godel_Agent，约 229 Stars（2026-10-17）。

核心架构图与解释

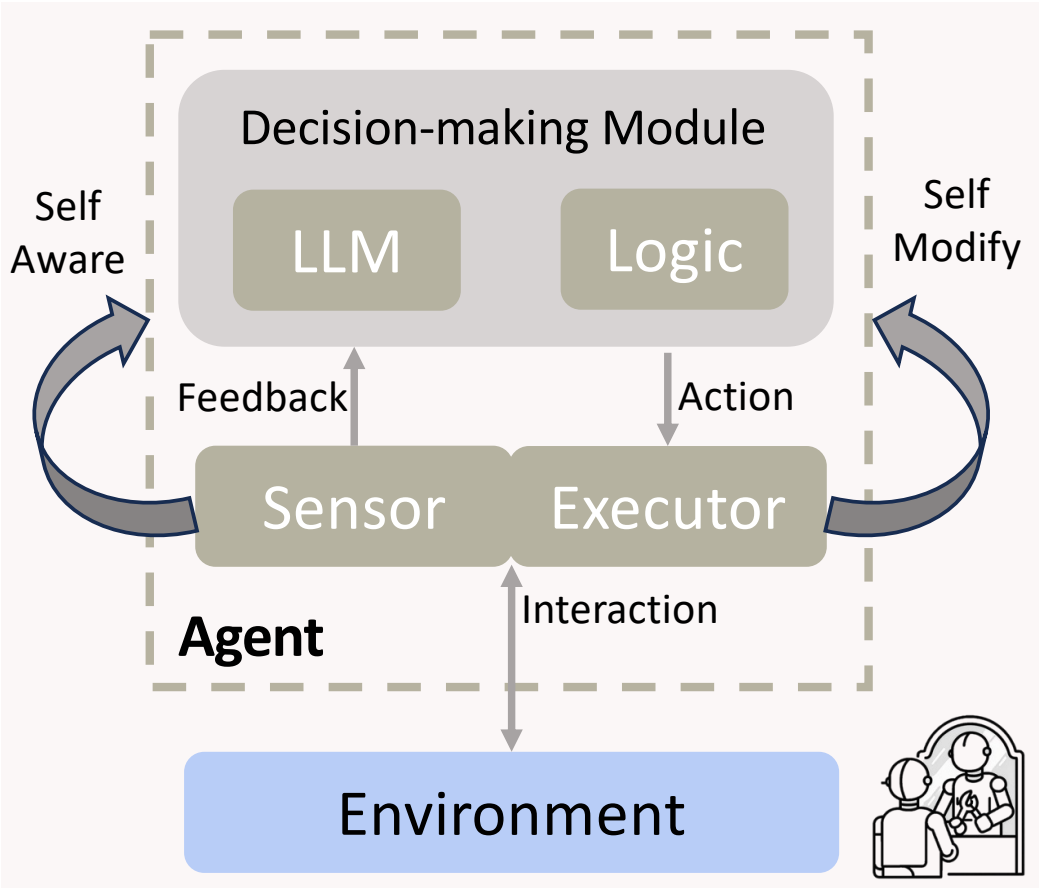


图 3.1: 固定策略、元学习更新与自指式自修改之间的区别。

原图来源：论文 arXiv:2410.04444。

固定架构的 Agent 使用预先设计好的策略；元学习系统可以根据反馈更新策略，但更新策略的算法本身保持不变。Gödel Agent 将两者都纳入可修改对象：Agent 读取自身代码，根据目标和效用反馈生成动作，再执行代码更新。修改后的策略与改进算法继续参与后续迭代，这构成其递归自我改进机制。

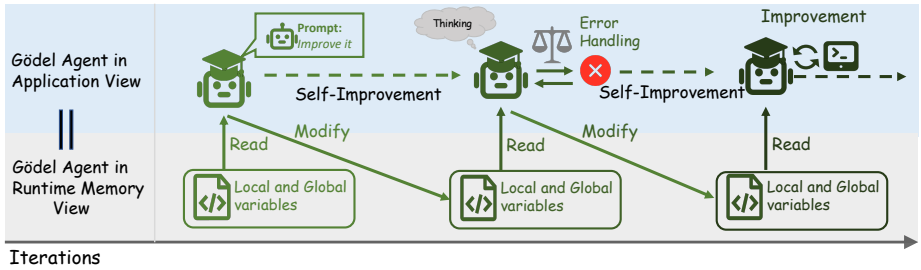


图 3.2: 通过运行时自省与代码修改实现自指式改进。

原图来源：论文 arXiv:2410.04444。

图中的 Runtime Memory View 表示 Agent 可读取的运行时代码视图。论文实现通过 monkey patching 等机制，使生成的更新能够作用于当前运行逻辑。决策函数根据策略、自身代码、效用反馈和目标提出动作，执行函数再根据动作类型读取代码、获取环境反馈、应用更新或递归调用改进过程。

数学表示

令 π_t 为时刻 t 的任务策略，I_t 为负责自我改进的算法，r_t 为环境反馈。普通的元学习更新写作

$$\pi_{t+1} = I(\pi_t, r_t), \qquad r_t = U(\mathcal{E}, \pi_t). \tag{3.1}$$

这里的 I 固定不变。Gödel Agent 则允许更新策略和改进算法本身：

$$\pi_{t+1}, I_{t+1} = I_t(\pi_t, I_t, r_t, g), \qquad r_t = U(\mathcal{E}, \pi_t), \tag{3.2}$$

其中 g 是高层优化目标。递归性来自 I_t 可以产生 I_{t+1}，而不仅仅产生新的任务策略 π_{t+1}。

Recursive Self-Improvement of Gödel Agent（Gödel Agent 的递归自我改进）

```
输入：初始策略 pi_0、初始决策函数 f_0、目标 g、环境状态 E、效用函数 U、SELF_INSPECT
输出：优化后的策略 pi 与框架代码 s

s = SELF_INSPECT()                  # 读取包括本算法在内的 Agent 代码
r = U(E, pi_0)                      # 计算初始效用
(pi, s) = SELF_IMPROVE(E, pi_0, s, r, g)

function SELF_IMPROVE(E, pi, s, r, g):
    actions = f_0(pi, s, r, g)      # 生成动作序列
    for action in actions:
        (pi, s, r) = EXECUTE(E, pi, s, r, action)
    return (pi, s)

function EXECUTE(E, pi, s, r, action):
    switch action.name:
        case "self_state":
            s = SELF_INSPECT()
        case "interact":
            r = U(E, pi)
        case "self_update":
            (pi, s) = action.code
        case "continue_improve":
            (pi, s) = SELF_IMPROVE(E, pi, s, r, g)
    return (pi, s, r)
```

决策函数 f_0 根据当前策略、自身代码、效用反馈和目标生成动作序列；执行函数根据动作名称读取自身代码、与环境交互、应用代码更新，或递归调用改进过程。

3.2 SICA：自我改进的 Coding Agent

论文：A Self-Improving Coding Agent（SICA，arXiv:2504.15228）
开源参考：MaximeRobeyns/self_improving_coding_agent，约 407 Stars（2026-10-17）。

核心架构图与解释

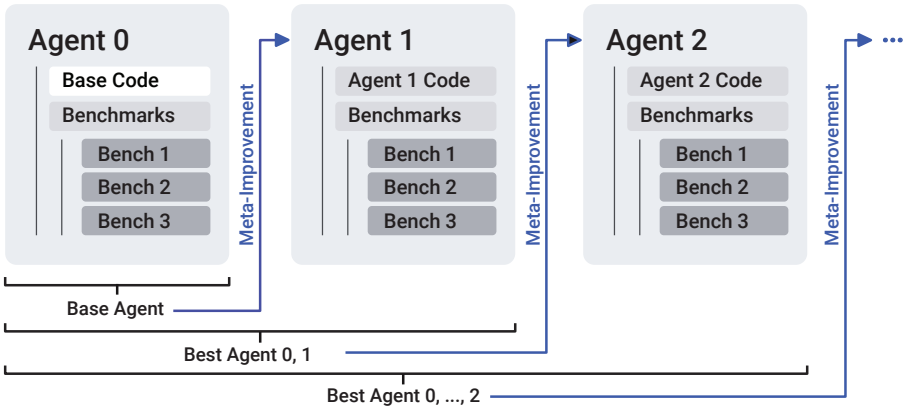


图 3.3: SICA 的版本迭代与自我改进循环。

原图来源：论文 arXiv:2504.15228。

SICA 将 Coding Agent 本身作为进化对象。每轮先在固定基准上评估当前版本，并把分数与运行记录写入 Archive；随后从历史版本中选择得分最高的 Agent，让它读取版本档案和评价结果，修改自身代码并生成下一版。新版本作为候选加入后续迭代，历史版本则继续保留。

可修改的对象不只是 Agent 为外部任务编写的代码，也包括它自身的工具、提示、上下文管理和任务执行逻辑。其核心循环是“评估版本—选择历史最佳—生成新版本”，而不是简单地让上一轮版本覆盖自身。

数学表示

令 A_i 为第 i 轮的 Agent 版本，B 为固定任务集，p_i 为版本在任务集上的评价结果。论文算法每轮先评估版本，再选择当前档案中得分最高的版本生成下一代：

$$\hat{i} = \underset{j \in \{0,\ldots,i\}}{\arg\max}\, p_j. \tag{3.3}$$

Self-Referential Agent Improvement（自指式智能体改进）

```
输入：基准集合 B = {b_1, ..., b_m}，迭代次数 n
输出：改进后的 Agent A_n

初始化 Agent A_0
for i = 0, 1, ..., n-1:
    在基准集合 B 上评估 A_i，记录分数 p_i
    best_index = argmax(p_j, j = 0, ..., i)
    A_{i+1} = 运行 A_best_index，结合历史版本 A_0...A_i
              及其分数 p_0...p_i，生成下一版 Agent
return A_n
```

每轮先评估当前版本，再从已评估的历史版本中选出分数最高者，令其读取版本档案与历史分数并生成下一版 Agent。因此，下一轮的修改起点不一定是上一轮的产物，历史高分版本仍可被重新选中。

3.3 Darwin Gödel Machine：开放式自我进化

论文：Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents（arXiv:2505.22954）

开源参考：jennyzzt/dgm，约 2.4k Stars（2026-10-17）。

核心架构图与解释

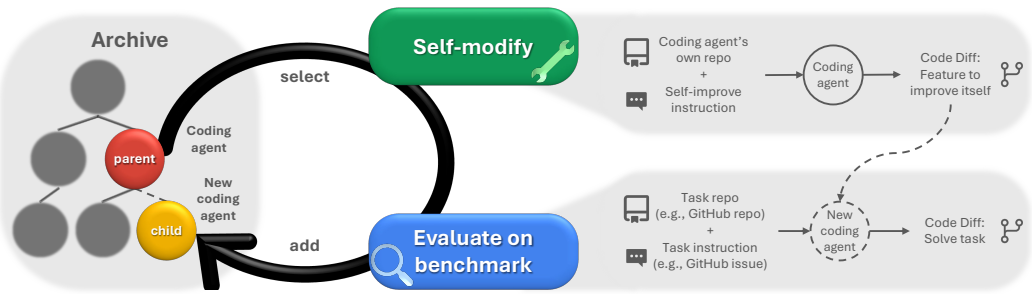


图 3.4: DGM 将自修改、任务评价与候选 Agent 档案连接为开放式进化循环。

原图来源：论文 arXiv:2505.22954。

DGM 通过任务评价检验自修改候选，而不是要求每次修改都先经过形式化证明。系统从初始 Agent 建立 Archive；每轮选择档案中的父代，由父代修改自身代码，再在基准任务上评价新版本。通过有效性检查的候选及其分数被加入 Archive，成为后续搜索的潜在起点。

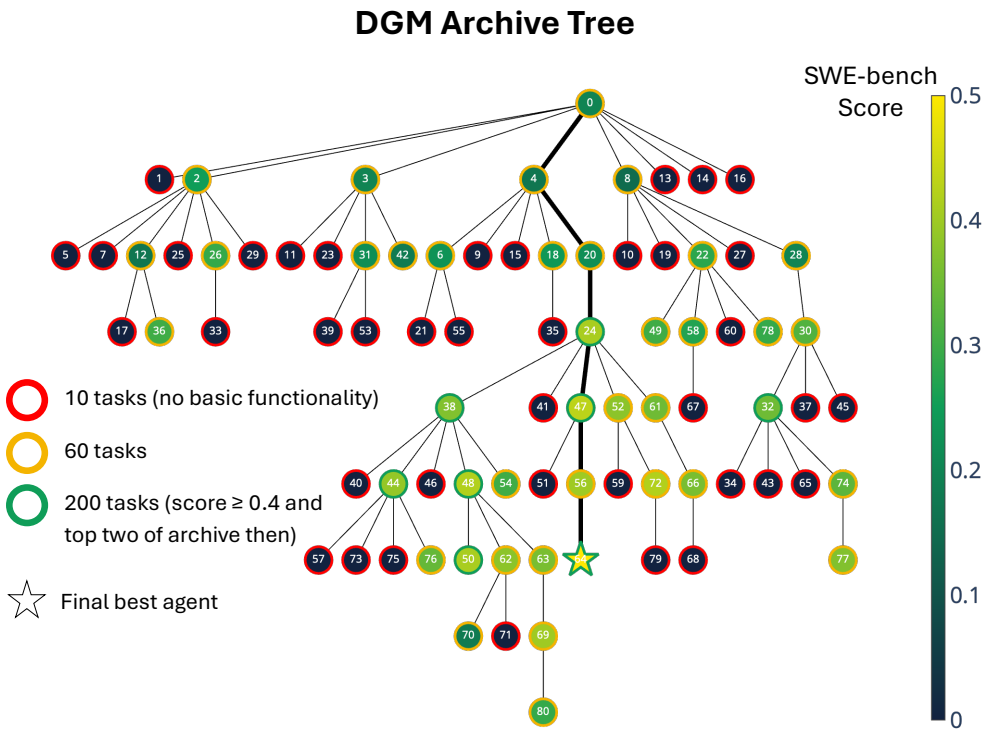


图 3.5: DGM 的历史候选档案与进化树。

原图来源：论文 arXiv:2505.22954。

Archive 的作用不只是记录当前最佳版本，还在于保留不同的搜索起点。即使某个候选暂时没有最高分，它仍可能包含值得继续扩展的机制；后续候选可以从不同历史版本分支产生。因此，DGM 维护的是不断扩展的候选树，而非只沿着单一最佳版本前进的线性链。

Darwin Gödel Machine（DGM）

```
输入：初始编码 Agent g_0、基准集合 B、最大迭代次数 T
输出：Agent 档案 A

s_0 = evaluate(g_0, B)
A = {(g_0, s_0)}
for t = 1, ..., T:
    P = SelectParents(A)                # 选择父代 Agent
    for parent in P:
        child = parent.modify(parent)   # 父代修改自身
        score = evaluate(child, B)      # 在基准集合上评价
        if child.is_valid():
            A = A union {(child, score)} # 保留有效候选
return A
```

父代选择决定档案中哪些版本获得继续扩展的机会；自修改产生候选，外部任务评价提供反馈，有效性检查排除无法继续执行或修改代码的候选。档案保留多个历史版本，因此搜索可以沿不同分支继续，而不必始终覆盖为单一当前版本。

3.4 Huxley-Gödel Machine：以进化潜力指导搜索

论文：Huxley-Gödel Machine（HGM，arXiv:2510.21614）
开源参考：metauto-ai/HGM，约 416 Stars（2026-10-17）。

核心架构图与解释

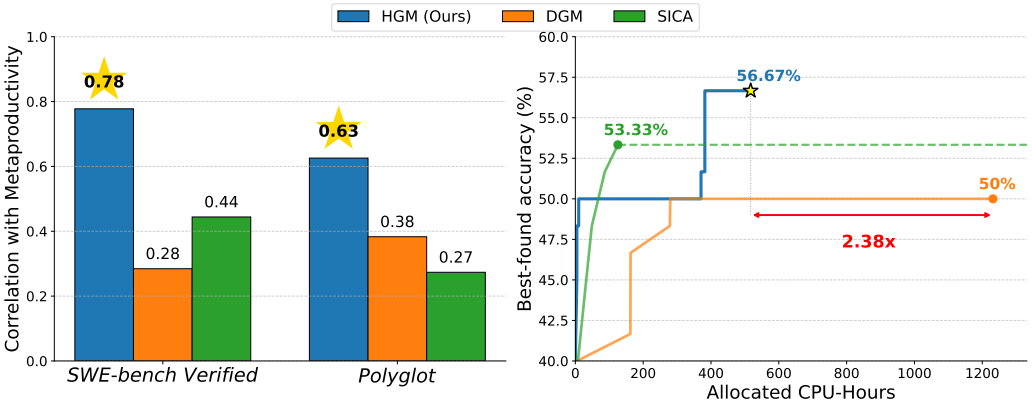


图 3.6: HGM 的核心动机：当前任务表现与长期自我改进能力可能失配。

原图来源：论文 arXiv:2510.21614。

HGM 将父代选择的依据从单个 Agent 的当前表现扩展到其后代的进化表现。论文称当前任务表现与长期自我改进潜力之间的差异为 Metaproductivity–Performance Mismatch：当前分数较高的 Agent 未必能产生有价值的后代，当前表现普通的 Agent 也可能开启更有潜力的分支。因此，只按当前分数选择父代可能过早丢弃有价值的搜索方向。

数学表示

HGM 使用 Clade-Metaproductivity（CMP）近似衡量以某个节点为根的整条后代子树的生产力。令 $C(a)$ 为以 Agent a 为根的 clade，$n^{C}_{\text{success}}(a)$ 和 $n^{C}_{\text{failure}}(a)$ 分别为该子树所有节点的成功与失败评估数，则估计量为

$$\widehat{\text{CMP}}(a) = \frac{n^{C}_{\text{success}}(a)}{n^{C}_{\text{success}}(a) + n^{C}_{\text{failure}}(a)}. \tag{3.4}$$

它聚合了节点及其后代的评价证据。一个父代即使自身表现普通，只要其后代分支持续产生更好的 Agent，就可能获得较高的 clade 估计值。

Huxley–Gödel Machine（HGM）

```
输入：初始 Agent a_0、扩展参数 alpha、最终选择分位数 epsilon
初始化以 a_0 为根的进化树 T
为每个节点初始化自身及其子树的成功/失败计数

while 计算预算尚未耗尽：
    if |T| <= n^alpha 且存在可扩展的父节点：
        # 扩展阶段
        对每个可扩展节点 a，采样 S_C(a) ~ Beta(tau(1+n_success^C(a)),
        tau(1+n_failure^C(a)))
        a_star = S_C(a) 最高的节点
        c = 对 a_star 执行自修改
        将 c 加入 T
    else：
        # 评价阶段
        对仍有未评测任务的每个 Agent a，采样 S(a) ~ Beta(tau(1+n_success(a)),
        tau(1+n_failure(a)))
        a_star = S(a) 最高的 Agent
        为 a_star 分配一个基准任务并记录结果
        更新 a_star 的成功/失败计数
        更新 a_star 及其所有祖先的子树计数
return argmax_{a in T_B} I_epsilon(1 + n_success(a), 1 + n_failure(a))，其中 T_B 为
    ↪ 已评测 Agent 集合
```

HGM 将“选择哪个 Agent 继续产生后代”和“选择哪个 Agent 接受新的基准评价”分开处理。两个阶段都使用 Beta 分布采样来处理成功率估计的不确定性：扩展阶段依据子树统计选择父代，评价阶段依据节点自身统计分配任务。评价结果会同时更新该节点及其祖先的子树统计，因此后代表现能够影响祖先在后续扩展中的机会。

3.5 HyperAgents：让改进机制本身成为可修改对象

论文：HyperAgents（arXiv:2603.19461）
开源参考：facebookresearch/HyperAgents，约 2.7k Stars（2026-10-17）。

核心架构图与解释

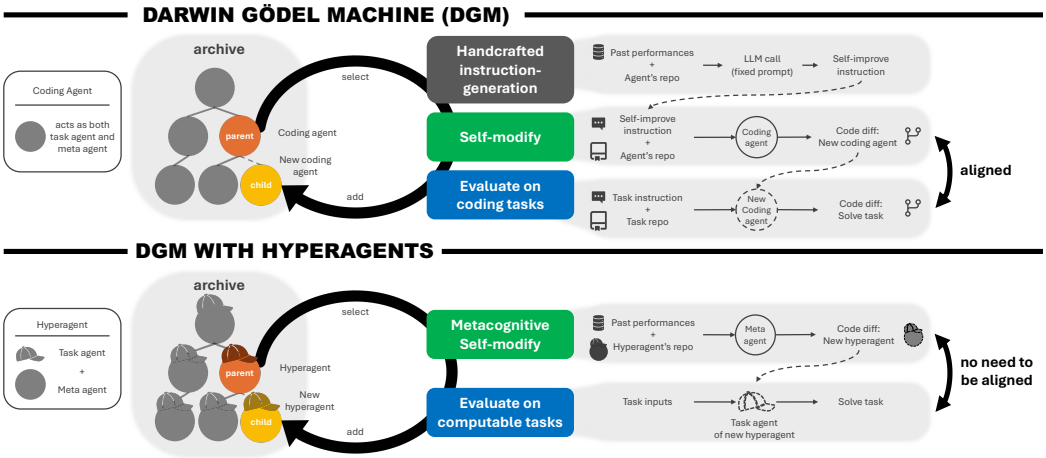


图 3.7: DGM 与引入 HyperAgents 后的系统差异。

原图来源：论文 arXiv:2603.19461。

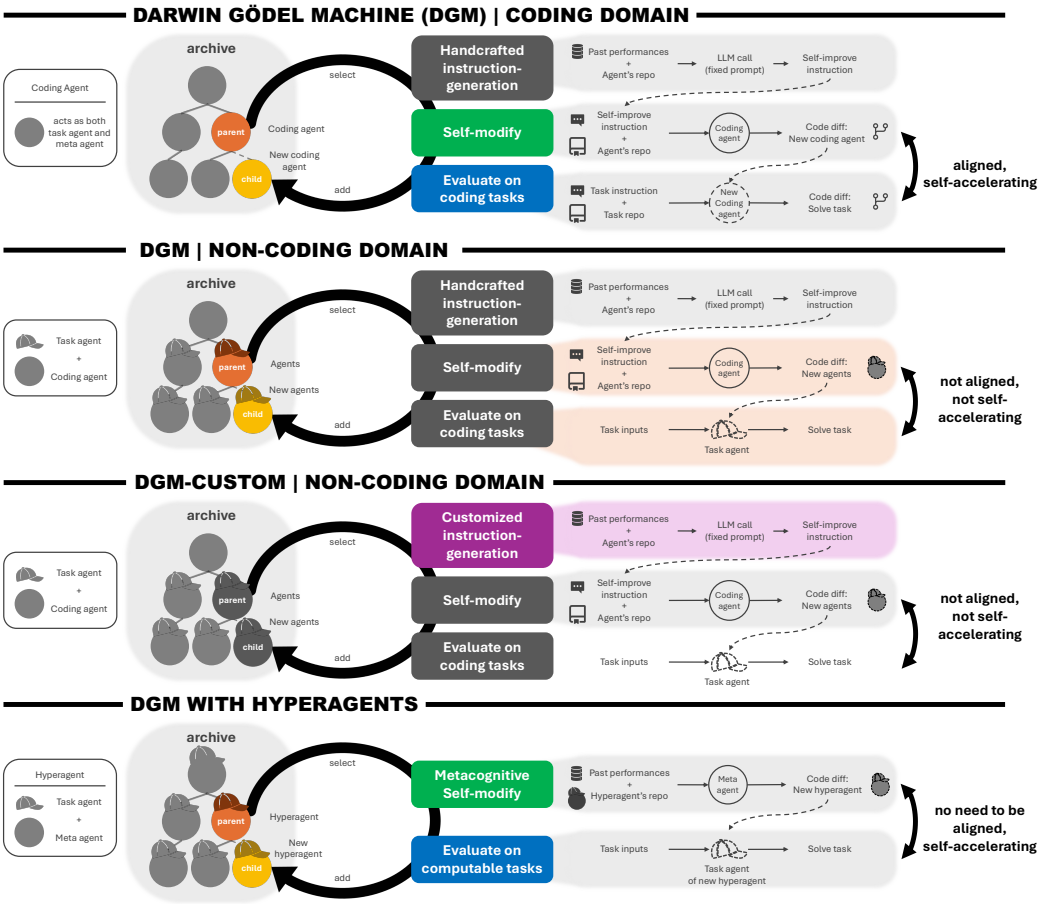


图 3.8: HyperAgents 的详细概念结构。

原图来源：论文 arXiv:2603.19461。

在 DGM 的编码任务设置中，同一个 Coding Agent 同时承担任务执行与候选生成的角色。编码能力提升通常也有助于修改编码 Agent 自身；但在论文评审或机器人奖励设计等非编码任务中，任务能力与自修改能力未必同步提升。

HyperAgents 将 Task Agent 与 Meta Agent 合并为一个可修改的程序，称为 hyperagent。它既能修改任务求解逻辑，也能修改负责提出后续修改的元层机制。DGM 的 Archive 和开放式分支探索仍被保留，变化在于生成未来改进方案的机制本身也进入了可进化范围。

DGM-H with Modifiable Parent Selection（带可修改父代选择的 DGM-H）

```
输入：初始 Agent a_0、任务集合 T、最大迭代次数 N
输出：带有评价分数的 Agent 档案 A

s_0 = Evaluate(a_0, T)
A = {(a_0, s_0)}
for t = 1, ..., N:
    latest = A 中最近加入的 Agent
    P = latest.SelectParents(A)          # 由最新 Agent 的选择机制选父代
    for parent in P:
        child = parent.Modify(parent, A) # 修改 Agent，也可修改父代选择机制
        score = Evaluate(child, T)
        if IsValid(child):
            A = A union {(child, score)}
return A
```

该算法的关键不只是生成新候选，而是由最新加入档案的 Agent 决定如何选择父代；修改过程也能调整这一选择机制。因而，进化对象不仅包括任务执行逻辑，还包括影响后续搜索方向的元层策略。